

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS
(NIVEL 2 (INTERMEDIO))

Módulo 02: Ordenamiento, Búsqueda y Big-O

«El Diccionario por la Mitad y Divide y Vencerás»

Análisis de complejidad y eficiencia computacional: Notación Big-O, Búsqueda Lineal $O(n)$ vs Búsqueda Binaria $O(\log n)$, y algoritmos de ordenamiento (MergeSort, QuickSort).

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Intermedio | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Notación Asintótica Big-O y Complejidad	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Diagrama de Búsqueda Binaria (Divide & Conquer)	Pág. 5
Capítulo 3	Implementación Práctica en Python: Búsqueda Binaria y QuickSort en Python	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas en Algoritmos de Búsqueda	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender cómo escala el tiempo de ejecución a medida que el tamaño de entrada (n) crece hacia el infinito.



Competencia Práctica

Implementar búsqueda binaria y entender la estrategia 'Divide y Vencerás' de QuickSort frente a BubbleSort.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Notación Asintótica Big-O y Complejidad

En software la velocidad no se mide en segundos, sino en cómo crece el número de operaciones en función del volumen de datos (n).

☀ Metáfora Central: El Diccionario por la Mitad y Divide y Vencerás

Si buscas una palabra en un diccionario de 1,000 páginas hojeando página por página (búsqueda lineal), puedes tardar 1,000 pasos. Si abres el diccionario por la mitad exacta y descartas la mitad irrelevante (búsqueda binaria), encontrarás la palabra en solo 10 pasos.

Principios Teóricos y Modelo Mental

Escalas de Complejidad: $O(1)$ Constante < $O(\log n)$ Logarítmica < $O(n)$ Lineal < $O(n \log n)$ Casi-lineal < $O(n^2)$ Cuadrática.

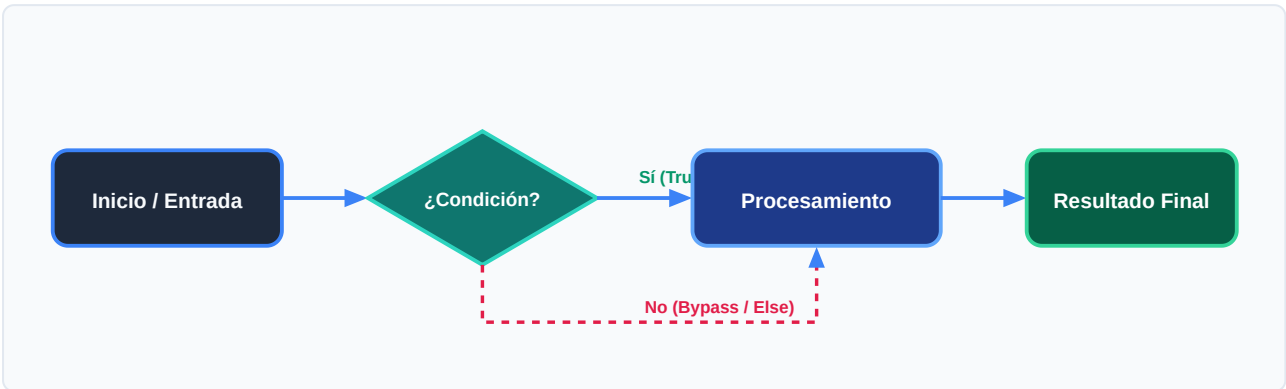
Búsqueda Binaria requiere que la colección esté previamente ordenada para garantizar reducción del espacio de búsqueda a la mitad.

⚡ Regla de Oro en Python

Evita los bucles anidados innecesarios: dos bucles anidados sobre n elementos convierten un algoritmo de $O(n)$ a $O(n^2)$.

2. Diagrama de Búsqueda Binaria (Divide & Conquer)

Estrategia de reducción logarítmica del intervalo de búsqueda con punteros low, mid y high.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Calcula el punto medio: $\text{mid} = (\text{low} + \text{high}) // 2$.	$\text{low}=0, \text{high}=\text{n}-1, \text{mid}$
2. Evaluación	Compara el elemento en 'mid' con el objetivo buscado.	Evalúa igualdad
3. Transformación	Si $\text{objetivo} < \text{array}[\text{mid}]$, descarta la mitad derecha ajustando $\text{high} = \text{mid} - 1$.	Espacio reducido al 50%
4. Retorno / Salida	Si $\text{objetivo} > \text{array}[\text{mid}]$, descarta la mitad izquierda ajustando $\text{low} = \text{mid} + 1$.	Repite hasta converger

Visualización Mental

La búsqueda binaria puede encontrar un registro entre 4 mil millones de elementos en tan solo 32 comparaciones.

3. Búsqueda Binaria y QuickSort en Python

Implementación idiomática de búsqueda binaria y particionado recursivo QuickSort:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
def busqueda_binaria(lista: list[int], objetivo: int) -> int:
    low, high = 0, len(lista) - 1
    while low <= high:
        mid = (low + high) // 2
        if lista[mid] == objetivo:
            return mid # Encontrado
        elif lista[mid] < objetivo:
            low = mid + 1
        else:
            high = mid - 1
    return -1 # No existe

def quicksort(arr: list[int]) -> list[int]:
    if len(arr) <= 1:
        return arr
    pivote = arr[len(arr) // 2]
    izq = [x for x in arr if x < pivote]
    centro = [x for x in arr if x == pivote]
    der = [x for x in arr if x > pivote]
    return quicksort(izq) + centro + quicksort(der)
```

Análisis del Código Fuente

Búsqueda Binaria $O(\log n)$ combinada con QuickSort $O(n \log n)$ basado en listas por comprensión.

4. Gotchas en Algoritmos de Búsqueda

Errores clásicos al implementar algoritmos de búsqueda y ordenamiento:

⚠ Gotcha Frecuente (Trampa de Principiante)

Ejecutar búsqueda binaria sobre una lista no ordenada; produce falsos negativos y resultados erráticos.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
desordenada = [9, 1, 5, 2]  
busqueda_binaria(desordenada, 5) # ¡Falla!
```

✓ Patrón Pythonic / Correcto

```
ordenada = sorted(desordenada)  
busqueda_binaria(ordenada, 5) # Retorna índice  
correcto
```

🛡 Consejo de Resiliencia en Producción

Python utiliza Timsort (híbrido de MergeSort e InsertionSort) en `lista.sort()`, el cual tiene complejidad garantizada $O(n \log n)$.

5. Resumen Ejecutivo y Conclusiones

Comprendes el impacto exponencial de los algoritmos en el rendimiento y dominas las técnicas de ordenamiento y búsqueda.



Logro Alcanzado en esta Clase

Capacidad para analizar la complejidad temporal de algoritmos y optimizar cuellos de botella.

Notas del Instructor y Sigüientes Pasos

En el siguiente módulo exploraremos la Recursividad y la Programación Dinámica con Memoización.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Compara el tiempo de ejecución en segundos entre una búsqueda lineal y una binaria sobre 1 millón de elementos con el módulo `time`.